

Lab Day 26 — Node.js Basics Recap

Week 6 · 45 minutes · Individual lab

Overview

This lab reinforces Node.js fundamentals covered in Day 24 (introduction) and Day 26 (deep recap). You will work inside the **lab-starter/** folder. Complete each task in order. Run every file with `node <file>.js`.

Prerequisite	You completed
Day 22	MongoDB basics
Day 23	Express + Mongoose REST API
Day 24	Node.js introduction
Day 26 lecture	Node recap (event loop, fs, http, modules)

Setup

- Open terminal in **Week 6/Day 26/lab-starter/**
- Verify Node: `node --version` (v18+ recommended)
- No npm install required — tasks use only built-in Node modules

Task 1 — Event Loop Prediction (8 min)

File: `task1-event-loop.js`

Before running the file, write the expected console output order on paper. Then run the file and compare. In a comment at the bottom of the file, explain WHY each line prints in that order (sync vs microtask vs macrotask).

```
// Predict first, then run:
console.log('1: sync');

setTimeout(() => console.log('4: timeout'), 0);

Promise.resolve()
  .then(() => console.log('2: promise'))
  .then(() => console.log('3: chained promise'));
```

Expected answer: 1: sync → 5: sync end → 2: promise → 3: chained promise → 4: timeout

Task 2 — path & fs (10 min)

File: `task2-fs-path.js`

Complete the TODOs to: (1) build a file path with `path.join`, (2) write a JSON object to `data/users.json`, (3) read it back asynchronously with `fs/promises`, (4) print the parsed array length.

- Create the `data/` folder if it does not exist
- Use `JSON.stringify(data, null, 2)` for pretty output
- Handle errors with `try/catch`

Task 3 — process & Environment (5 min)

File: `task3-process.js`

Complete the script to print: Node version, platform, current working directory, and a custom env var `LAB_NAME`. Run twice: once normally, once with `set LAB_NAME=NodeLab` (Windows) or `LAB_NAME=NodeLab node task3-process.js` (macOS/Linux).

Task 4 — EventEmitter Logger (8 min)

File: task4-events.js

Build a tiny logger class that extends `EventEmitter`. It should have a `log(level, message)` method that emits a 'log' event with `{ level, message, timestamp }`. Register a listener that prints:
[INFO] 2026-... Your message.

```
const { EventEmitter } = require('events');

class Logger extends EventEmitter {
  log(level, message) {
    // TODO: emit 'log' event with { level, message, timestamp: new Date().toISOString() }
  }
}

const logger = new Logger();
logger.on('log', (entry) => {
  // TODO: print formatted line
});

logger.log('INFO', 'Server started');
logger.log('WARN', 'Low disk space');
```

Task 5 — Raw HTTP Server (12 min)

File: task5-http-server.js

Create a server on port 4000 using only the built-in `http` module (no Express). Requirements:

- GET / → JSON `{ "ok": true, "message": "Node HTTP lab" }`
- GET /time → JSON with current ISO timestamp
- Any other route → status 404, JSON `{ "error": "Not found" }`
- Set Content-Type: `application/json` on every response
- Test with browser or `curl http://localhost:4000/time`

Bonus (+5): Read the port from `process.env.PORT` with fallback 4000. This mirrors how your Express apps from Day 23 should be configured.

Task 6 — Modules Refactor (2 min)

Files: `utils/format.js` + update `task4-events.js`

Move the log formatting logic into `utils/format.js` using `module.exports`. Import it in `task4`. This mirrors how Express projects split routes, models, and helpers.

Submission Checklist

- All 5 task files run without errors
- Task 1 has a comment explaining event loop order

- data/users.json created by Task 2
- HTTP server responds correctly on / and /time
- utils/format.js exported and used in Task 4/6

Questions? Raise your hand. Stuck for more than 5 minutes on one task — skip ahead and return later.